National Institute of Technology Raipur

Department of Information Technology



Project Report

on

Decentralised Multispectral UAV Swarms for Search and Rescue: Early-Fusion Perception, Delay-Tolerant Networking and Energy-Aware Self-Organisation

Submitted by:

1. Devesh Agarwal (23118030)
2. Sankalp Mittal (23118089)
3. Naman Chaudhary (23118069)

Guided by:

Dr. Rayanoothala Praneetha Sree

Branch: Information Technology

Course: B. Tech.

Semester: 7

Contents

1 Introduction **3**

1.1 Search and Rescue with UAV Swarms . . . 3
1.2 Motivation and Problem Statement . . . 3
1.3 Objectives . . . 3
1.4 Contributions . . . 4
1.5 Organisation of this Report . . . 4

2 Background and Related Work **4**

2.1 Comparison of Existing Approaches . . . 4
2.2 Literature Review by Theme . . . 5
2.3 Research Gap . . . 5

3 System Model and Problem Formulation **6**

3.1 Arena, UAVs and Victims . . . 6
3.2 Radio Model . . . 6
3.3 Objective and Constraints . . . 6
3.4 State, Decisions and Information . . . 6

4 Proposed System and Algorithms **6**

4.1 System Architecture . . . 6
4.2 Multispectral Victim Detection . . . 7
4.3 Wireless Channel Model . . . 7
4.4 Delay-Tolerant Alert Dissemination . . . 8
4.5 Energy-Weighted Territory Partitioning (EW-CVT) . . . 8
4.6 Dual-Threshold Hysteresis . . . 10
4.7 Claim-and-Yield Sector Assignment . . . 10
4.8 Lease-Based Failure Detection and Failover . . . 10
4.9 Control Loop . . . 10
4.10 Mission State Machine . . . 11
4.11 Complexity . . . 12

5 Implementation **12**

5.1 Software Stack . . . 12
5.2 Mission Simulator . . . 12
5.3 Switches for Ablation . . . 13
5.4 Ablation and Reporting Pipeline . . . 13
5.5 Repository Layout . . . 13

6 Experimental Setup **14**

6.1 Simulation Environment . . . 14

6.2 Evaluation Metrics . . . 14
6.3 Baseline Methods . . . 14
6.4 Detector Conditions . . . 14
6.5 Statistical Protocol . . . 15

7 Results and Analysis **15**
7.1 Detection Performance . . . 15
7.2 Reference Mission Performance . . . 15
7.3 Ablation Study . . . 16
7.4 Networking Results . . . 18
7.5 Comparison with Baseline Methods . . . 19
7.6 Failure Recovery Analysis . . . 20
7.7 Scaling with Swarm Size . . . 21

8 Discussion **22**
8.1 What the Evidence Supports . . . 22
8.2 The Price of Decentralisation . . . 22
8.3 Epidemic versus Bounded Forwarding . . . 22
8.4 Perception under Degraded Conditions . . . 22

9 Limitations and Future Work **22**
9.1 Limitations . . . 22
9.2 Future Work . . . 23

10 Conclusion **23**

References **24**

1 Introduction

1.1 Search and Rescue with UAV Swarms

The first hours after an earthquake, a flood or a landslide largely determine who is found alive, and the survival rate drops sharply with time [1]. Ground teams are slow and often cannot enter the affected area. Small unmanned aerial vehicles (UAVs) can fly over a wide area much faster, and a swarm of them can cover it faster still, since every aircraft searches a different part of the area at the same time. Cameras on board can show people to a rescue coordinator, and a coordinator who sees a list of positions can dispatch teams where they are needed.

Using a swarm in a real disaster is harder than the idea suggests. This project addresses three practical problems that arise together.

- **Perception in poor conditions.** A visible-light (RGB) camera performs poorly in shadow, at dusk and in smoke, where a thermal (long-wave infrared) camera can still show a warm person. A thermal camera, in turn, struggles when people and their surroundings have nearly the same temperature. A detector that uses both can work in more conditions than either alone.
- **Communication.** Over areas several kilometres wide the UAVs are mostly out of radio range of each other and of the ground station. A design that assumes a connected network would deliver almost nothing. Findings must be stored, carried and handed over when two UAVs, or a UAV and the ground station, happen to meet.
- **Coordination without a planner.** A single ground planner is a single point of failure and needs links that disaster areas rarely offer. If the UAVs decide for themselves, however, they may search the same sector twice, leave a sector unsearched, or overload a UAV whose battery is low. If a UAV is lost, the others do not know it: in flight, silence is normal and tells nothing.

1.2 Motivation and Problem Statement

Perception, communication and control are normally studied one at a time. They depend on each other, however. What a UAV can decide depends on what it has heard, and what it hears depends on the radio link and on where the other UAVs have flown. A detector that finds every person is of no use if the alert never reaches the operator, and a delay-tolerant network is of no use if the UAV that carries the alert is the one that is lost.

The problem studied in this project is therefore as follows: *design and evaluate a swarm of UAVs that searches a large area, detects people with a multispectral camera, delivers the alerts to a ground station over an intermittent radio network, and re-organises itself without a central planner when batteries differ or a UAV is lost.*

1.3 Objectives

The objectives of the project are:

1. to build a four-channel RGB and thermal early-fusion person detector and measure it in daylight, at night and under thermal washout;
2. to design a delay-tolerant alert mechanism with per-recipient buffers and store-carry-forward relaying, and to measure it under a physical radio model;
3. to design a decentralised coordination layer that works on stale local views: an energy-weighted territory partition, a hysteresis gate against jitter, claim-and-yield sector assignment, and a failure detector that treats silence as normal;

4. to evaluate every component with a controlled ablation and with comparisons against simpler alternatives, and to report the effects with confidence intervals, including the unfavourable ones.

1.4 Contributions

The main contributions are:

1. An integrated, decentralised architecture for search and rescue with intermittently connected swarms. It combines early-fusion perception (YOLO11), delay-tolerant alert dissemination and energy-aware coordination, and the three parts run on every UAV.
2. A coordination layer that works on stale local views: an energy-weighted centroidal Voronoi tessellation (EW-CVT) with dual-threshold hysteresis, claim-and-yield sector assignment, and a lease-based failure detector that releases the work of an overdue UAV only at base.
3. A delay-tolerant alert mechanism with per-recipient QoS-1 buffers and epidemic store-carry-forward relaying.
4. A controlled evaluation over 1,620 paired simulated missions, with a component-wise ablation and a comparison with spray-and-wait, buffering only, a static partition and a nearest-bidder auction without failover.

1.5 Organisation of this Report

Section 2 reviews related work and the gap. Section 3 defines the system model and the problem. Section 4 describes the proposed system and its algorithms, and Section 5 describes the implementation. Section 6 explains the experimental setup, and Section 7 gives the results. Section 8 discusses them, Section 9 lists the limitations and the future work, and Section 10 concludes.

2 Background and Related Work

2.1 Comparison of Existing Approaches

Table 1 summarises the families of methods that bear on this project, with their usual strengths and the limitation that this project addresses.

Table 1: Families of related methods and the limitation addressed here.

Family	Strength	Limitation for a sparse swarm
Auction-based task allocation [2, 3]	Simple and distributed; good load balance	Assumes a connected network; failures are usually reported by the failing robot
Consensus [4]	Provable agreement	Needs repeated exchanges over connected links
Voronoi coverage control [5, 6]	Smooth, well-studied partition	Assumes agents see each other's positions; ignores energy unless weighted
Unreliable failure detectors [7, 8]	Formal treatment of suspicion	Rely on heartbeat arrival, which fails when silence is normal
Delay-tolerant routing [9, 10, 11]	Works without end-to-end paths	Trades delivery against copies; usually studied apart from control
Multispectral detection [12, 13, 1]	Better detection in poor light	Seldom evaluated inside a mission with a radio model

2.2 Literature Review by Theme

UAVs for search and rescue

Small UAVs have been used to support wilderness and disaster search since the field trials of Goodrich et al. [14]. For several robots, coordinated exploration and task allocation have been studied by Burgard et al. [15] and by Gerkey and Matarić [2], and auction-based methods decide which robot searches which part of the area. Grøntved et al. [3] adapt such an auction to search and rescue and assume a fully connected network. Giacomossi et al. [16] re-auction the sectors of a failed UAV, but the failure is reported by the UAV itself and the simulation has no radio losses. Other methods reach agreement by consensus [4]. Our setting resembles flying ad hoc networks with sparse, intermittent links [17].

Multispectral person detection

Combining visible and thermal images is known to improve pedestrian detection in poor light [12, 13]. Early fusion, in which the two modalities are stacked as input channels, needs little extra computation and is therefore suitable for onboard hardware [1]. We use it on top of the YOLO family [18, 19]. Speth et al. [1] released the multispectral aerial dataset that we use, and our detector follows their early-fusion design.

Coverage control

Dividing an area among agents with Voronoi cells and moving each agent to the centroid of its cell (Lloyd iteration) is a standard coverage method [20, 6, 5]. Weighting the distance by a quantity that belongs to each agent gives a weighted Voronoi diagram. A power diagram [21] subtracts the weight from the squared distance, and we divide by it instead. Energy-aware versions of Lloyd's method use such weights to give smaller regions to robots with less energy [22, 23]. Our weight represents the battery level in the same way, so we do not claim the weighting itself as new. In contrast to the classical setting, our agents act on outdated views received over a lossy link and resolve conflicts by claiming sectors, and we add hysteresis to the updates to avoid chattering when the inputs are noisy.

Failure detection

Deciding that a peer has failed when the channel is unreliable is the problem addressed by unreliable failure detectors [7]. Accrual detectors [8] rely on heartbeat arrival times, which does not suit a mesh in which silence is normal. Our sortie lease follows instead the lease idea of Gray and Cheriton [24]: a claim is valid until a stated time, and after that time it can be released.

Delay-tolerant networking

When end-to-end paths rarely exist, nodes can store messages and forward them when they meet other nodes [9]. Epidemic and spray-and-wait routing [10, 11] trade the delivery rate against the number of copies and the buffer use. We use epidemic forwarding with a buffer cap, and Section 7 compares it with spray-and-wait and with buffering only in the same simulator. The per-recipient buffers follow DMLT [25], and the radio model is a simple log-distance channel [26]. The simulation platform builds on FlyNetSim [27].

2.3 Research Gap

The works above treat perception, networking and control separately, and the coordination methods assume either a connected network or failures that are announced. The gap that this project fills is a swarm that (i) perceives with fused RGB and thermal frames, (ii) delivers its alerts over a mostly disconnected radio network, and (iii) re-organises itself from stale local views, with a failure detector that does not mistake silence for failure. We evaluate the three parts together.

3 System Model and Problem Formulation

3.1 Arena, UAVs and Victims

A team of N UAVs launches from a common base and searches an $L \times L$ arena that is divided into K sectors with centres c_s. UAV i has a position, a battery level $B_i \in [0, B_{\max}]$ and a set of claimed sectors. An unknown set of victims lies in the sectors. A UAV that scans a sector takes one multispectral frame of it and raises an alert for each victim that the detector finds. The ground control station (GCS) is a passive sink at the base: it receives alerts and plans nothing.

3.2 Radio Model

The radio links are lossy and time-varying. The packet error rate (PER) between two UAVs depends on their distance (Section 4.3), so the communication graph is mostly disconnected in flight. Two UAVs can exchange messages only when they meet, and the UAVs can reach the GCS at base.

3.3 Objective and Constraints

The objective is to maximise the coverage of the arena and the share of victims whose alert reaches the GCS (the operator recall), while keeping the mission time and the alert latency short. Three constraints apply:

1. a UAV must return to base before its battery falls below a 15% reserve;
2. there is no central planner, so every decision may use only the information that the UAV has itself received;
3. a UAV may be lost without notice, and the others cannot observe the loss directly.

3.4 State, Decisions and Information

Each UAV holds a local *view* of the swarm that is built from the status messages it has received: the last advertised position, committed generator and weight of each peer, the sectors that each peer claims and the time at which it should return, and the set of sectors that are known to be scanned. The view is usually outdated. The decisions that a UAV makes from its view are: which sectors to claim, in which order to visit them, when to return, and which peers to suspect of being lost. Because all UAVs use the same rules on views that differ only slightly, the swarm organises itself.

4 Proposed System and Algorithms

4.1 System Architecture

Fig. 1 gives an overview. Every UAV runs the same three components. *Perception* turns a multispectral frame of the sector below into victim detections. *Mesh communication* sends alerts and periodic status messages over a lossy radio link, buffers what could not be sent, and carries received alerts onward. *Control* chooses which sectors to search, from the UAV's own battery and its current view of its peers, and builds a route. The interfaces are narrow: alerts go from perception to the mesh, status messages update the control view of the peers, and control hands a route to flight. The aircraft and the radio are simulated, whereas the detection, networking and control logic is the code under test.

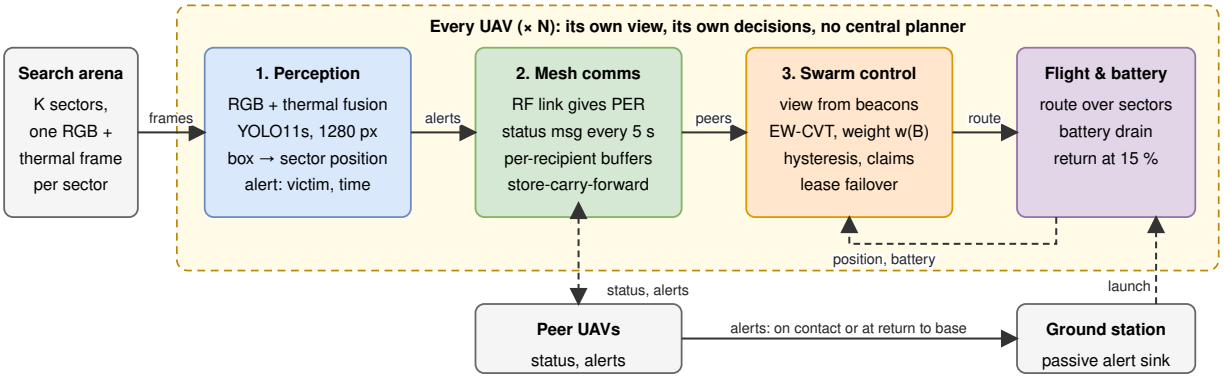


Figure 1: System architecture. Solid arrows: data flow; dashed arrows: feedback, radio exchange and launch.

4.2 Multispectral Victim Detection

The visible (RGB) and thermal long-wave infrared frames are stacked into a four-plane tensor, and the first convolution of the detector takes four input channels. The model is YOLO11s [19] with an input size of 1280 px, and the frames are 1280×937. It is trained on the NII-CU Multispectral Aerial Person Detection dataset [1] with a single person class. The thermal kernel is initialised from the pretrained red-channel kernel, which is the same technique as in the base paper, and training uses no colour jitter because colour has no meaning for a thermal plane.

Georeferencing. The centre of a detected box is mapped linearly into the bounds of the sector in which the frame was taken, which gives a world coordinate. This is sector-based georeferencing and not a camera model, so the localisation error in the simulation measures the detector and not the optics.

Fig. 2 shows the path from the two cameras to an alert. *Why early fusion.* Stacking the planes adds almost no computation to the first layer, so the detector is cheap enough for onboard use, and it lets the network use whichever plane is informative when the light is bad.

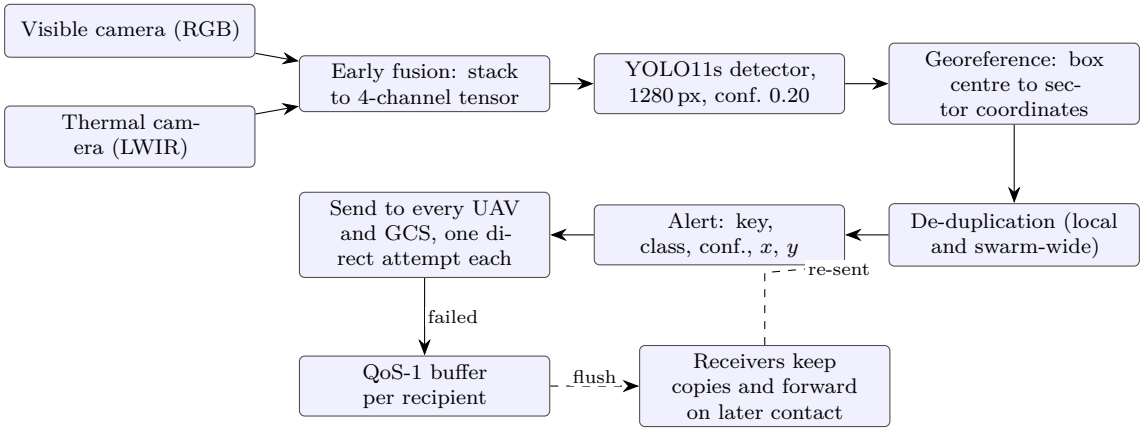


Figure 2: Per-UAV pipeline from sensors to the operator: perception (top) and alert handling (bottom).

4.3 Wireless Channel Model

The path loss between two UAVs at distance d follows a log-distance model with shadowing and obstruction:

$$PL(d) = PL(d_0) + 10\,n \log_{10} \tfrac{d}{d_0} + X_\sigma + \Delta_{\text{obs}}, \tag{1}$$

where $PL(d_0) = 20 \log_{10} f_{\mathrm{MHz}} - 27.55$, $n = 2.5$, $d_0 = 1\,\mathrm{m}$, X_σ is log-normal shadowing with $\sigma = 4\,\mathrm{dB}$, and $\Delta_{\mathrm{obs}} = 15\,\mathrm{dB}$ when the link is obstructed (a random 10% chance beyond 50 m) and 0 otherwise. The noise floor is $-90\,\mathrm{dBm}$. The signal-to-noise ratio $\mathrm{SNR} = P_{tx} - PL - N_0$ maps to a packet error rate: PER = 0 above 25 dB, PER = 1 below 5 dB, and in between

$$\mathrm{PER} = 1 - \left(1 + e^{-0.5(\mathrm{SNR}-15)}\right)^{-1}. \tag{2}$$

Arenas of 2 km or more use 33 dBm at 2.4 GHz, which gives PER ≈ 0.5 at about 520 m. Smaller arenas use 30 dBm at 5 GHz, with PER ≈ 0.5 at about 220 m. In an arena that is several kilometres wide, the swarm is mostly disconnected in flight.

4.4 Delay-Tolerant Alert Dissemination

Alert generation and buffering

When a UAV detects a victim, it makes one direct attempt to reach every other UAV and the GCS. Each failed copy is stored in a *per-recipient QoS-1 buffer*, which is retained until it has been delivered. The buffer for a recipient is flushed when the link to that recipient has a PER below 0.2, which in practice happens when the UAV returns to base. A buffer with a capacity limit drops its oldest copy first.

Store-carry-forward relaying

This is epidemic forwarding [10]. A UAV that *receives* an alert also keeps a copy for each recipient that is not known to hold it, and it hands the copy over when it later meets that recipient with a good link. No UAV needs to be in the right place at the right moment, and an alert survives the loss of its finder. On contact, only the copies that the other side lacks are sent.

Fig. 3 shows what happens to one alert, step by step: the only network operation is a point-to-point attempt, and everything indirect is store-carry-forward.

Status messages

Every 5 s, each UAV broadcasts its position, its committed EW-CVT generator and weight, its state, the time at which its route should bring it home, its sector claims, and the set of sectors that it knows to be scanned. These messages are single-hop and are not buffered, since stale state is not worth delivering late. A receiver merges the scanned sets as grow-only sets, so the knowledge about coverage spreads transitively when UAVs meet.

4.5 Energy-Weighted Territory Partitioning (EW-CVT)

Let B_i be the battery level of UAV i and $B_{\max}$ the capacity. The UAV has the weight

$$w_i = (B_i / B_{\max})^\alpha. \tag{3}$$

A sector with centre c_s is preferred by the UAV with the smallest weighted distance

$$d(s, i) = \frac{\|c_s - g_i\|^2}{\max(0.01,\, w_i)}, \tag{4}$$

where g_i is the committed generator of the UAV. For $\alpha = 0$ the partition is a plain Voronoi diagram. A larger α shrinks the territory of low-battery UAVs, so the healthy UAVs do more of the work. The launch partition is the converged centroidal Voronoi tessellation, obtained by 15 Lloyd iterations: each generator is moved to the centroid of its cell, and this is repeated. Technically the result is a multiplicatively weighted Voronoi diagram and not a power diagram.

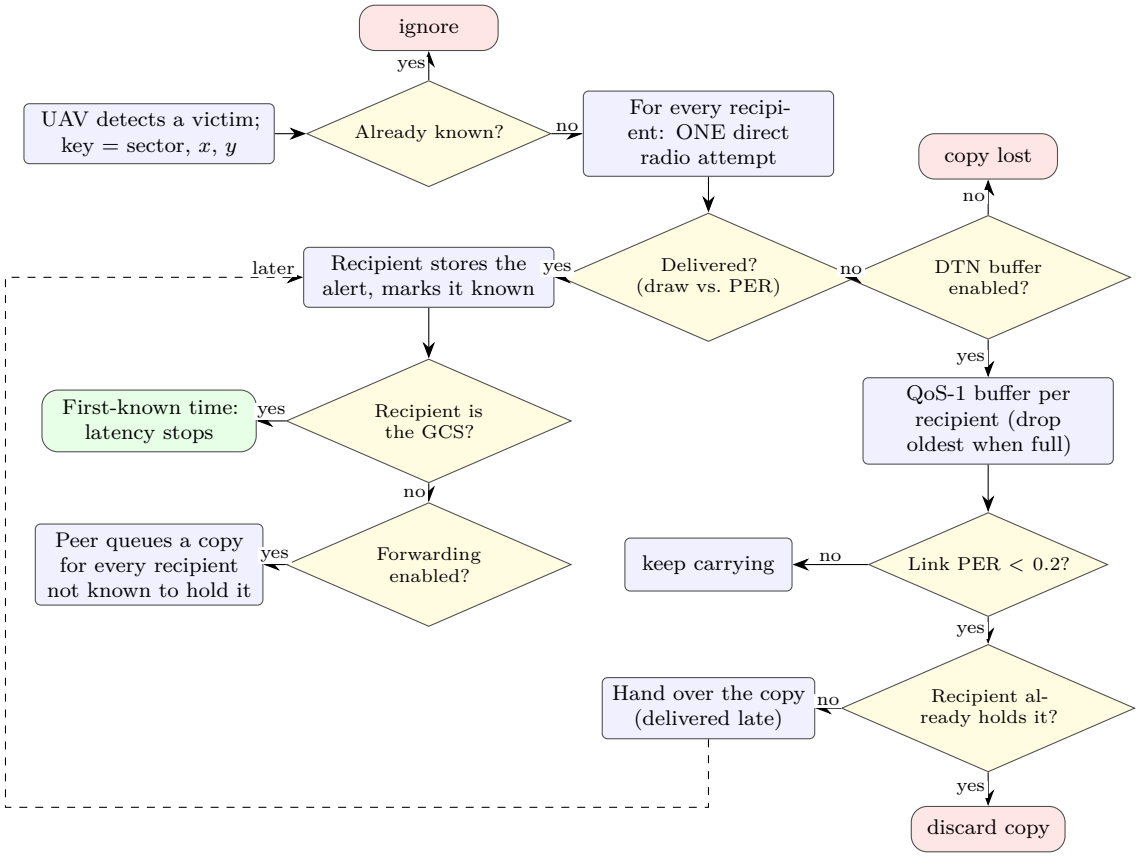


Figure 3: Packet delivery algorithm for a single alert: direct attempt, per-recipient buffering, store-carry-forward and contact-triggered hand-over.

4.6 Dual-Threshold Hysteresis

Every 5 s, each UAV computes the centroid C_i^* that its cell should have. It does so over the peers that it *believes* to be active, using their last advertised generators and weights. The UAV commits the new generator and weight only if

$$U_i = \big(\Delta B_i \geq \tau_\epsilon \ \wedge \ \|C_i^* - C_i\| \geq \tau_\delta\big) \ \vee \ \big(\mathcal{N}_i(t) \neq \mathcal{N}_i(t_k)\big), \tag{5}$$

with $\tau_\epsilon = 2\%$ of $B_{\max}$, $\tau_\delta = 5\,\text{m}$, and $\mathcal{N}_i$ the view of the live set held by the UAV. The first term requires both a real change in battery and a real movement of the centroid; the second term forces an update when the believed live set changes, for example after a failure. Fig. 4 shows the gate as a flowchart. Without this gate, noisy battery readings (here $\sigma = 1.5$ points) would commit a new partition at every tick, and the route would change continuously.

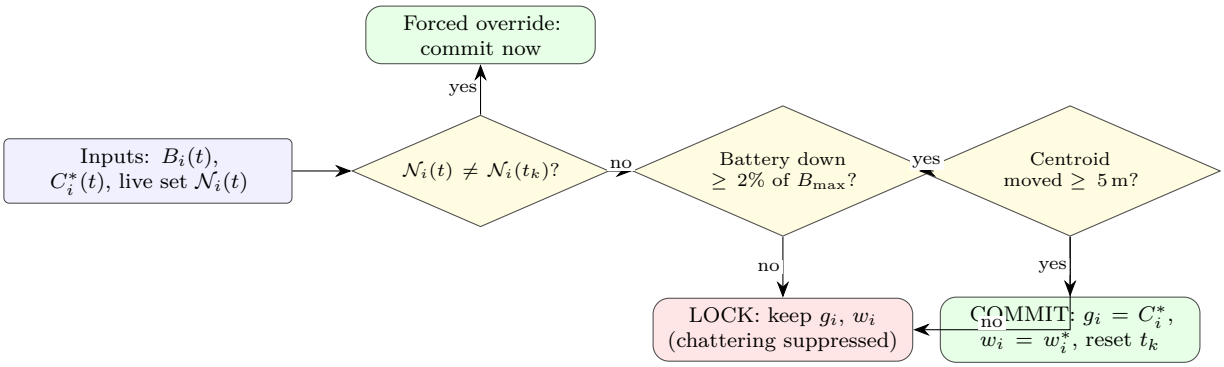


Figure 4: Dual-threshold hysteresis gate with the topology override.

4.7 Claim-and-Yield Sector Assignment

A UAV *claims* a sector if it is the preferred owner by $d(s, i)$ (Eq. 4) using committed values and no other UAV holds a valid claim on the sector. It keeps its existing claims unless a better claim (a smaller d, with ties broken by id) reaches it, and it then *yields*. It may take over the claim of a peer only if it would arrive well before the advertised arrival time of that peer. A UAV that has no claims left takes the nearest unclaimed sector. Routes are greedy nearest-neighbour tours that end at the launch pad. Because claims are sticky, the number of sector flips stays low even when the views differ.

4.8 Lease-Based Failure Detection and Failover

In flight, silence tells a UAV nothing about a peer, because the peer may be out of range or it may have landed. Instead, each peer advertises the time at which its route should bring it home (its *sortie lease*). A peer that is still missing *slack* seconds after this time (60 s by default) is *suspected* to be lost, and its claims are released. Since a lost peer looks like one that landed quietly, released work is taken only by UAVs *at base*, which have heard the landed peers. Landed UAVs listen for 10 s, split the work among themselves by $d(s, i)$, and relaunch if the battery allows. A suspicion is withdrawn when the peer is heard again. A UAV aborts and returns at 15% battery, and it is lost at 0%.

Fig. 5 shows the sequence after a loss. The slack is a trade-off. A short slack detects a loss sooner but suspects more UAVs that are fine (a landed UAV looks crashed to a peer that is still in flight). A long slack is almost free of false alarms but delays the failover by minutes. Section 7 quantifies this.

4.9 Control Loop

Algorithm 1 and Fig. 6 summarise one control tick of a UAV.

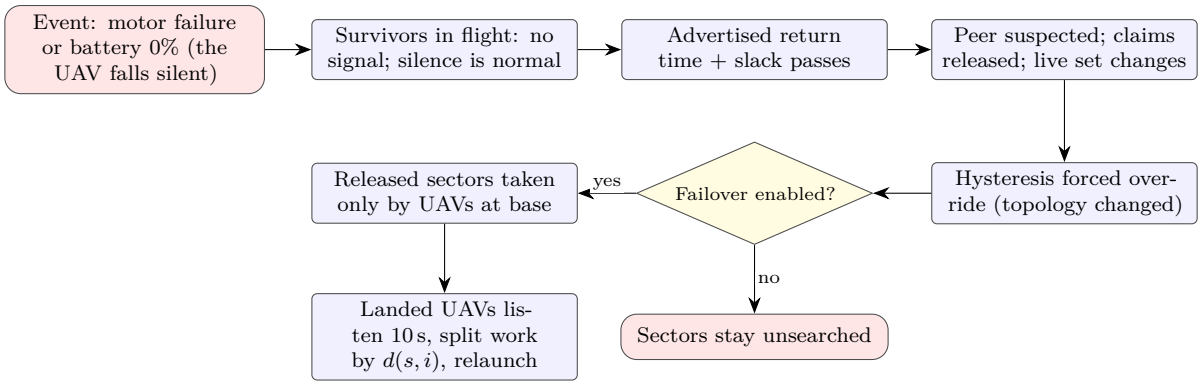


Figure 5: Failure and failover sequence. A peer that only landed looks like a dead one until a UAV reaches base.

Algorithm 1 Control tick of one UAV (every 5 s).

merge status messages received since the last tick into the view (peers, claims, scanned set)
for each peer j overdue at base by more than *slack* **do** mark j suspected; release its claims
end for
$\mathcal{N}_i \leftarrow$ self $\cup$ peers believed active (landed peers included only if self is at base)
read own battery B_i; $w_i^* \leftarrow (B_i/B_{\max})^\alpha$; $C_i^* \leftarrow$ EW-CVT centroid of my cell over $\mathcal{N}_i$
if U_i (Eq. 5) **then** commit $g_i \leftarrow C_i^*$, $w_i \leftarrow w_i^*$
end if
keep claims not beaten by a better claim; add free sectors that I prefer by $d(s,i)$
if no claims left **then** claim the nearest unclaimed sector; **if** at base, also share released work
end if
if claims changed **then** rebuild route; advertise route ETA, claims and scanned set
end if

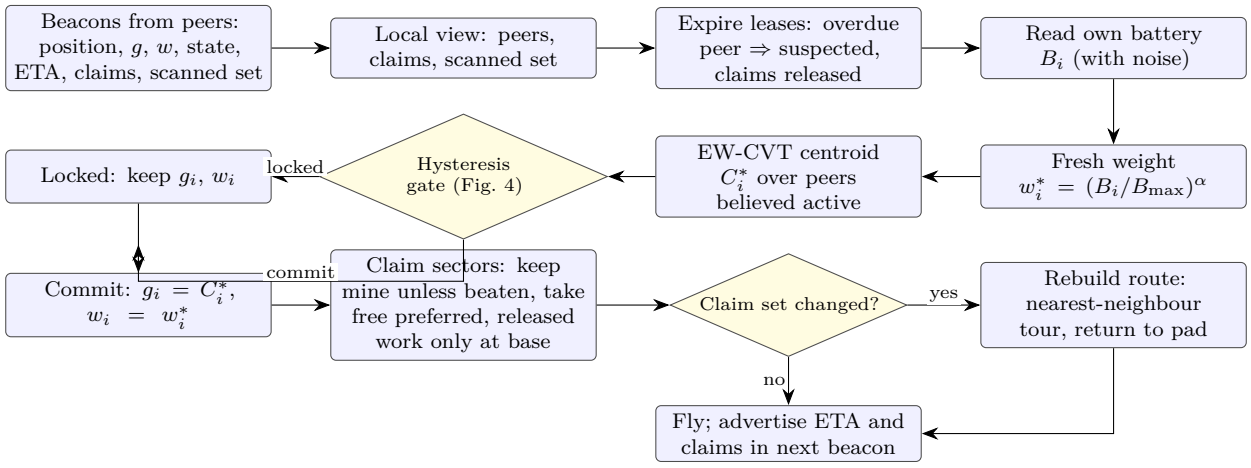


Figure 6: Control loop of one UAV, run every 5 s from its own view only.

4.10 Mission State Machine

A UAV is in one of five states: *TRANSIT* (flying to its first sector), *SEARCHING* (scanning its claimed sectors), *RETURNING* (heading home because its route is done or its battery has reached the 15% reserve), *LANDED* (at base, listening and sharing work), and *CRASHED* (battery empty or failed). Fig. 7 shows the transitions. The state is part of every status message. A landed UAV that has taken over released work and has enough battery relaunches into TRANSIT.

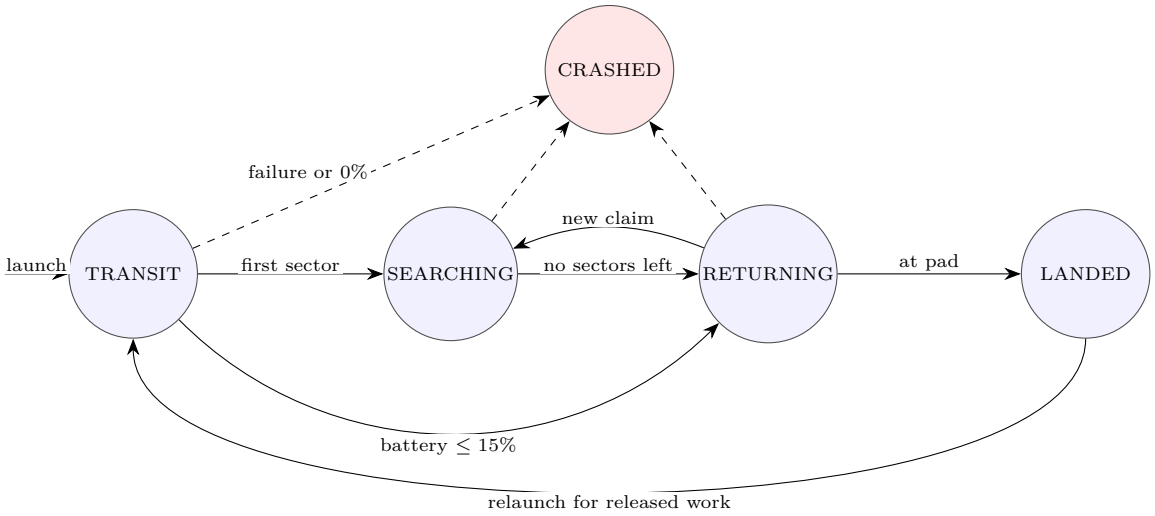


Figure 7: UAV state machine. A UAV keeps scanning sector after sector while SEARCHING. Dashed arrows lead to the absorbing CRASHED state; battery at or below 15% also forces RETURNING from SEARCHING.

4.11 Complexity

A control tick involves K sectors and $|\mathcal{N}_i| \leq N$ peers. Evaluating $d(s, i)$ for every sector and peer costs $O(KN)$, the centroid computation another $O(K)$, and the greedy route $O(K_i^2)$ for K_i claimed sectors. For $N = 6$ and $K = 25$ this is a few hundred operations every 5 s, which is negligible for onboard hardware. A status message is $O(K + N)$ in size because of the scanned set and the claims.

5 Implementation

5.1 Software Stack

The project is written in Python. The detector uses PyTorch and Ultralytics YOLO11 [19]. The project contains two layers.

- **Mission simulator** (`simulate_swarm_mission.py`, about 1,200 lines). A kinematic simulator with the classes `SimulatedUAV` and `SwarmSimulation`. It runs the full perception, communication and control logic described in Section 4 on real dataset frames, and it is the source of all reported numbers.
- **Flight and network platform** (`FlySim/`). A co-simulation platform derived from FlyNetSim [27] that couples ArduPilot software-in-the-loop aircraft with the ns-3 network simulator. It holds the swarm agent, the swarm network module, the hysteresis controller (`hysteresis_controller.py`), the publish/subscribe layers for the UAVs and the GCS, and utilities for RF checks and demonstrations.

5.2 Mission Simulator

Each sector of the arena shows one frame of the dataset, and the labels of that frame are the ground truth for the sector. The simulator advances in steps of one second. In every step each UAV moves along its route, drains its battery, scans the sector below if it has arrived (which calls the detector or reads its cache), generates and attempts alerts, and, every 5 s, runs a control tick and sends a status message. Links between UAVs and to the GCS are evaluated at every contact with the channel model of Section 4.3. A failure is injected by marking a UAV as lost at a given time without any message.

To make 1,620 missions affordable, detections are cached: a frame has to be passed through the detector only

once, and later missions that need the same frame read the result from the cache (`load_detection_cache` and `save_detection_cache`). A seed fixes the frame in each sector and every launch battery, so all variants of an experiment see identical worlds.

5.3 Switches for Ablation

Every component is a command-line switch, so an ablation changes one policy and nothing else. Table 2 lists the main switches.

Table 2: Command-line switches of the mission simulator used for the experiments.

Switch	Effect
`-drones, -arena, -grid`	Swarm size, arena width in metres, sectors per side
`-fail-drone, -fail-time`	UAV that is lost silently, and the time of the loss
`-alpha`	Exponent α of the EW-CVT weight
`-no-hysteresis`	Removes the dual-threshold gate
`-no-failover`	Released work is never taken over
`-static`	Fixed launch partition, no reassignment
`-no-dtn`	Removes the store-and-forward buffers
`-no-forwarding`	Removes store-carry-forward relaying
`-gcs-policy spray, -spray-copies`	Spray-and-wait relaying with L copies
`-allocation auction`	Nearest-bidder allocation (all weights equal to 1)
`-buffer-cap`	Maximum buffered copies per UAV
`-batt-min, -batt-noise`	Lowest launch battery, and noise of battery readings
`-beacon-interval`	Seconds between status messages
`-lease-slack`	Seconds past the advertised return before a peer is suspected

5.4 Ablation and Reporting Pipeline

The scripts in `ablation/` run the experiments and build the outputs: `run_ablation.py` runs the missions and writes `results/ablation/runs.json`; `perception_eval.py` evaluates the detector under the four input conditions; `make_report.py`, `make_readme.py` and `make_paper_assets.py` produce the HTML report, the results README, and the figures and tables of the paper. Every number in this report is generated by these scripts from the stored runs, which makes the study reproducible.

5.5 Repository Layout

- `simulate_swarm_mission.py`: mission simulator with perception, mesh and control.
- `ablation/`: experiment runner, perception evaluation, report and paper generators.
- `FlySim/`, `NetSim/`: ArduPilot and ns-3 co-simulation platform and its tools.
- `results/`: stored runs, figures, headline table and the HTML report.
- `docs/`: project overview, architecture diagrams, algorithms guide and how-to-run notes.
- `paper/`: the conference paper (LaTeX), the draw.io architecture diagram and the Overleaf archive.
- `report/`: this report.

6 Experimental Setup

6.1 Simulation Environment

Six UAVs search a 5×5 km arena that is divided into 25 sectors. Each sector shows one frame of the NII-CU multispectral aerial person detection dataset [1], whose labels serve as the ground truth (36 to 56 victims per world). The UAVs launch from a pad, fly at up to 50 m/s in transit and 32 m/s while searching, and drain 0.07 or 0.04%/s of battery above or below 40 m/s. The launch batteries are uniform between 20% and 100%, and the battery readings have a noise of $\sigma = 1.5$ points. In the loss scenario, UAV 2 fails silently at $t = 80$ s. The network experiments use arenas of 1.5, 3 and 5 km, and the scaling experiment uses 2 to 12 UAVs.

6.2 Evaluation Metrics

- **Coverage**: the share of sectors that were scanned at least once. **Mission time**: until completion or 1,500 s.
- **Precision** and **recall**: reported victims are matched to the ground truth within 15 m (one-to-one), and recall is taken over all victims in the arena, so victims in unsearched sectors count against the system.
- **Operator recall**: only the victims whose alert reached the GCS. **Alert latency**: the delay between detection and arrival at the GCS.
- **Failover latency**: the time from the injected failure to the first claim of one of its sectors by a survivor.
- **Duplicate scans**: sectors that were scanned more than once. **Sector flips**: claims given up to a better claim. **Route changes**: route rebuilds. **Commits**: updates approved by the hysteresis gate.
- **Copies per alert**: the number of UAVs that hold a copy at the end, a measure of traffic cost.

6.3 Baseline Methods

Each baseline replaces one policy in the same simulator, so the missions, the radio model and the seeds are identical.

- **Forwarding.** *Epidemic* (proposed): every UAV copies an alert to each UAV that it meets and that lacks one. *Spray-and-wait* [11]: each alert gets $L = 3$ copies, and the holder halves its copies at each hand-over and then waits to deliver directly. *Buffer only*: each alert is carried home by its finder, with no relaying.
- **Allocation.** *EW-CVT* (proposed): the full scheme of Section 4. *Auction without failover*: each sector is assigned to the nearest UAV (all weights equal to 1), and the sectors of a lost UAV are never reassigned. *Static partition*: the launch partition is fixed, with no reassignment at all.

6.4 Detector Conditions

The shipped model is run on 150 held-out validation frames (IoU 0.5, confidence 0.25). The *day* condition uses the frames unchanged. In the *night* condition, the RGB plane is multiplied by 0.12 and noise ($\sigma = 4$ grey levels) is added, which removes the visible cue and leaves thermal untouched. In the *thermal washout* condition, which emulates people and background at nearly the same temperature, the thermal plane is compressed to 25% of its contrast and noise ($\sigma = 5$) is added, with RGB untouched. Both are synthetic degradations of daytime frames, not recorded data. In the *RGB-only* and *thermal-only* conditions, the other plane is set to zero. This is modality-masked inference of one fused network and not a comparison of separately trained models.

6.5 Statistical Protocol

Paired seeds are used: a seed fixes the frame in each sector and every launch battery, so each variant sees the same worlds. The study has 1,620 paired missions (81 configurations). We report paired differences with 95% confidence intervals and claim no effect when an interval contains zero. Missions run until completion or 1,500 s.

7 Results and Analysis

7.1 Detection Performance

Table 3 shows the detector under the three conditions. The fused model reaches an AP50 of 99.8 in daylight. Fusion is about 11 points better than thermal only, so the visible plane adds information, while removing it at night costs only 5.5 points because thermal carries most of the detection. Thermal washout is the hardest case, because the network relies mainly on thermal: the recall falls to 0.39 while the precision stays at 0.99. The RGB-only case scores zero in every condition because the network was never trained without the thermal plane, so that number says nothing about the value of RGB, and trained single-modality baselines are still needed.

Table 3: Detection results on 150 held-out frames (AP in %; precision and recall at confidence 0.25).

	Day				Night				Thermal washout			
Input	AP50	AP75	Prec.	Rec.	AP50	AP75	Prec.	Rec.	AP50	AP75	Prec.	Rec.
Fused RGB + thermal (proposed)	**99.8**	**85.8**	**0.993**	**0.989**	**94.3**	**58.3**	0.916	**0.869**	**61.8**	**25.1**	**0.991**	**0.386**
Thermal only (RGB zeroed)	89.0	50.5	0.944	0.731	89.0	50.5	**0.944**	0.731	0.0	0.0	0.000	0.000

Comparison with the base paper

Table 4 compares the daytime result with the base paper [1], which reports YOLOv3 trained on the same dataset (Table 9 there, 485 aligned images with 891 people). Our fused YOLO11s model is ahead of their 4-channel model at AP50 (99.8 against 97.9) and at AP75 (85.8 against 76.9), and, like theirs, it is clearly ahead of the RGB-only and ensemble variants. The comparison is indicative only: we evaluate 150 of the 485 frames (275 people), the architecture (YOLO11s at 1280 px against YOLOv3 at 960×704) and the schedule differ, so part of the gap comes from the newer detector and not from the fusion. AP50:95 was not computed, and the checkpoint was selected on the same split. Latencies are not comparable (CPU here, a drone-class GPU in [1]).

Table 4: Comparison with Speth et al. [1] (AP in %). YOLO11s: 150 of 485 frames.

Model	Input	AP50	AP75
YOLOv3 [1]	RGB	92.4	44.5
YOLOv3 [1]	Thermal (FIR)	97.5	72.7
YOLOv3 [1]	RGB + FIR ensemble	97.3	54.7
YOLOv3 [1]	RGB + FIR, 4-channel	97.9	76.9
YOLO11s (proposed)	RGB + FIR, 4-channel	**99.8**	**85.8**

7.2 Reference Mission Performance

Without a failure, the decentralised swarm covers 100% of the arena in 353±31 s (mean ± s.d.). Precision, recall and F1 are all 0.980±0.015, where the whole-arena recall counts the victims in unsearched sectors. The

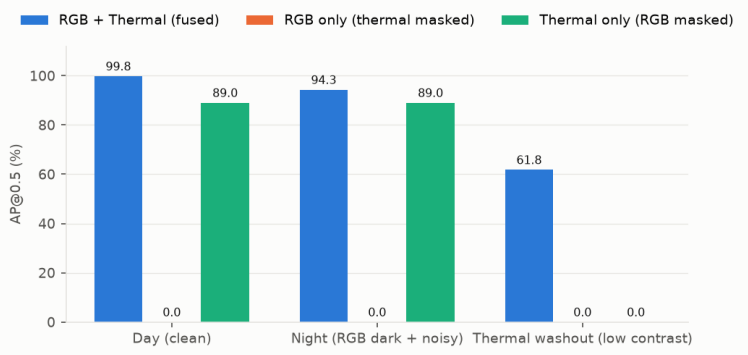


Figure 8: AP50 of the detector by input modality and condition.

mean localisation error is 1.45±0.15 m, the first victim is detected after 8.6±7.2 s, and the operator learns of 100% of the reports. No sector is scanned twice, and the minimum final battery is 18.8±8.3% of capacity.

7.3 Ablation Study

Table 5 gives the effect of removing each component, and Fig. 9 shows the coverage over time and the network effect.

Table 5: Ablation: paired change from the full system (mean ± 95% CI). Network rows: 5 km arena.

Component removed: measure	Full	Ablated	Change
Failover (UAV lost): coverage	98.4%	87.4%	-11.0 ± 2.6
whole-arena recall	0.962	0.840	-0.122 ± 0.032
Hysteresis†: sector flips	2.5	13.5	$+11.0 \pm 3.5$
route changes	4.8	25.4	$+20.6 \pm 6.4$
mission time	362 s	392 s	$+30 \pm 25$ s
EW weighting (α=0): min. battery	18.8%	17.9%	-0.8 ± 0.7
DTN buffer (5 km): operator knows	98.0%	3.4%	-94.7 ± 1.2
swarm-wide consensus	100%	22.1%	-77.9 ± 1.3
Forwarding (5 km): alert latency	103 s	109 s	$+7 \pm 2$ s

† per-tick re-planning test configuration, $\sigma = 2\%$ (see text).

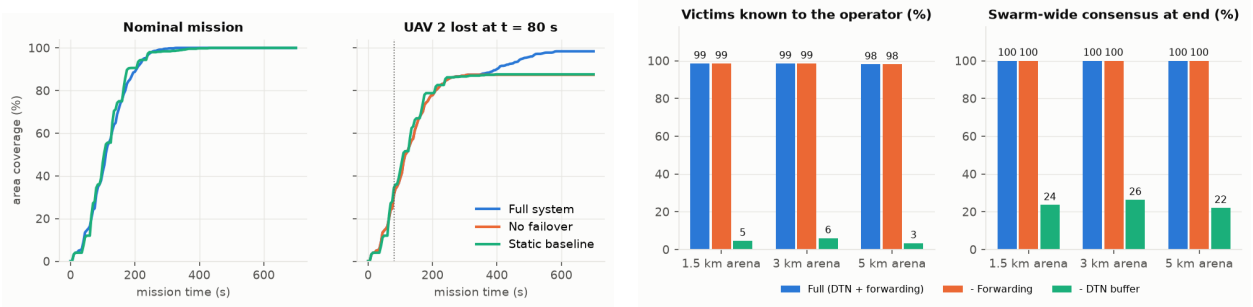


Figure 9: Left: coverage over time, nominal and with UAV 2 lost at 80 s. Right: victims known to the operator, by arena size.

Failover

Failover matters when a UAV is lost. The coverage after the loss is 98.4% with failover and 87.4% without it, since without failover the sectors of the lost UAV are never searched. The whole-arena recall falls from 0.962 to 0.840.

Hysteresis

Hysteresis reduces churn but not flight time. In the full decentralised system it has almost no measurable effect, because the claims are already sticky (0.2 versus 0.4 sector flips), and this is a statement about the planner and not evidence that the gate is useless. It matters when a global planner recomputes the partitions from fresh readings at every tick. Table 6 and Fig. 10 show that hysteresis then cuts the commits about tenfold and the sector flips by a factor that grows with the noise, with the mission time unchanged within the intervals.

Table 6: Hysteresis under battery-sensor noise (per-tick re-planning test).

Noise σ (%)	Flips ON	Flips OFF	Commits ON	Commits OFF	Time ON (s)	Time OFF (s)
0	1.8	5.2	32	294	377	389
1	2.1	9.6	31	295	372	388
2	2.5	13.5	31	296	362	392
4	4.1	23.4	30	300	364	400

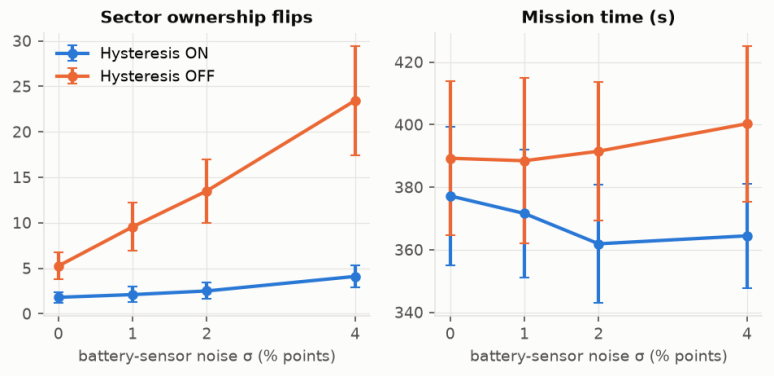


Figure 10: Hysteresis suppresses chattering as the sensor noise grows.

Energy weighting

EW weighting is a trade-off. When α is swept from 0 to 3 (Table 7, Fig. 11), the minimum final battery rises from 15.6% to 20.3% and the low-battery aborts fall from 0.95 to 0.30 per mission, but the mission becomes longer (377 to 488 s) because the healthy UAVs carry more of the work. Coverage is 100% at every α. The parameter is a dial between speed and energy safety, and not a free improvement.

Table 7: Effect of the EW-CVT exponent α (mean $\pm$ 95% CI).

α	Min. final battery (%)	Mission time (s)	Coverage (%)	Low-battery aborts
0	15.6±6.5	377±94	100.0	0.95±0.89
0.5	16.1±7.1	349±42	100.0	0.90±0.85
1	16.7±7.0	358±27	100.0	0.90±0.85
2	18.3±6.9	427±71	100.0	0.45±0.76
3	20.3±8.8	488±114	100.0	0.30±0.47

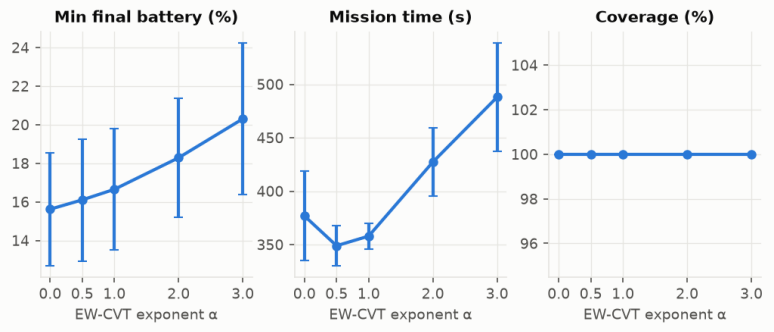


Figure 11: Time versus energy reserve as the exponent α is varied.

7.4 Networking Results

Under the radio model, direct links reach only a few hundred metres, so the swarm is mostly disconnected in flight and the first-attempt delivery to the GCS is only 3 to 6% in every arena. Store-and-forward queues the alerts in the buffer of the UAV and flushes them when it returns to base, which turns a mostly offline swarm into one that the operator can rely on. Table 8 shows that removing the buffer lowers the share of victims known to the operator from about 98% to between 3% and 6%, and the swarm-wide consensus from 100% to about 22 to 26%.

Table 8: Network ablation without a failure (mean $\pm$ 95% CI).

Arena	Variant	Operator recall	Alert latency (s)	Consensus
1.5 km	Full (DTN + forwarding)	0.985±0.013	34±4	1.000
	without forwarding	0.985±0.013	41±5	1.000
	without DTN buffer	0.046±0.028	–	0.236
3 km	Full (DTN + forwarding)	0.985±0.013	41±4	1.000
	without forwarding	0.985±0.013	59±7	1.000
	without DTN buffer	0.060±0.033	–	0.263
5 km	Full (DTN + forwarding)	0.980±0.015	103±13	1.000
	without forwarding	0.980±0.015	109±12	1.000
	without DTN buffer	0.034±0.019	–	0.221

A UAV is lost before it returns

If the finder is lost, its alerts reach the operator only if a peer that received them returns to base, and forwarding is what makes that possible. With the loss, forwarding raises the reports known to the operator from 0.976 to 0.989 at 1.5 km and from 0.973 to 0.993 at 3 km, and cuts the latency from 66 to 46 s at 3 km. At 5 km the finder's own return is early enough that both variants reach 1.000.

Buffer capacity

Epidemic forwarding stores many copies, so the capacity of the buffer matters. With a cap of 5, 15, 40 and 120 copies per UAV (3 km, UAV lost), the reports known to the operator are 0.225, 0.415, 0.834 and 0.993, and the copies dropped fall from 389 to 11. A cap of about 120 copies loses nothing in this scenario, and a small cap fails badly.

7.5 Comparison with Baseline Methods

Forwarding policies

Table 9 and Fig. 12 compare the forwarding policies. Epidemic forwarding has the lowest alert latency in every scenario, but it also holds the most copies. At 3 km, its latency is 7.6±1.7 s below spray-and-wait (paired difference with 95% confidence interval) and 18 s below buffering only, and it keeps about 1.9 more copies per alert than spray-and-wait. When UAV 2 is lost, the operator recall is 0.978 with epidemic forwarding, 0.967 with spray-and-wait and 0.958 with buffering only, and the latency gap to spray-and-wait grows to about 10 s. Otherwise the operator recall is the same for all policies.

Table 9: Forwarding policies (mean ± 95% CI). Lat.: alert latency; Copies: per alert.

	1.5 km		3 km		5 km		3 km, UAV lost	
Policy	Lat. (s)	Copies	Lat. (s)	Copies	Lat. (s)	Copies	Lat. (s)	Copies
Epidemic (proposed)	**34.3**±2.1	4.12	**41.0**±1.8	4.30	**102.9**±6.0	3.39	**46.3**±3.3	3.52
Spray-and-wait	36.4±2.5	2.38	48.6±2.2	2.37	106.3±5.3	2.14	56.4±4.8	2.29
Buffer only	41.4±2.3	1.00	59.2±3.3	1.00	109.5±5.7	1.00	66.4±4.3	1.04

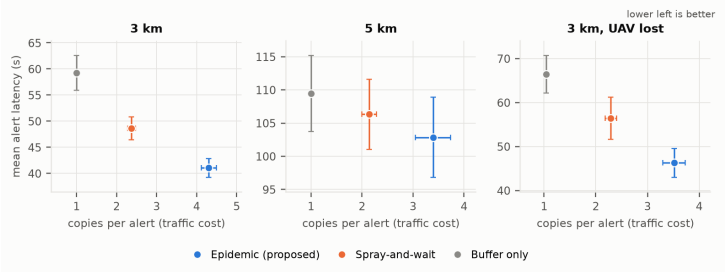


Figure 12: Alert latency versus copies per alert (bars: 95% CI).

Allocation policies

Table 10 and Fig. 13 compare EW-CVT with a nearest-bidder auction without failover and with a static partition. Without a loss, all three reach full coverage. EW-CVT has no duplicate scans (0.00 against 0.90 and 0.10) and the highest minimum battery (18.8% against 17.4% and 17.9%), but it is slower than the auction (353 against 334 s). After a loss, the coverage is 98.4% with failover against 89.2% and 87.6% for the auction and the static partition, and the arena recall is 0.96 against 0.87. The price is a longer mission, 554 s against about 340 s, because the survivors take over the work of the lost UAV. We credit the failover, and not the weighting, with the coverage gain.

Table 10: Allocation policies, nominal and with UAV 2 lost. Dup.: duplicate scans.

	Nominal				UAV 2 lost			
Policy	Cover. %	Time (s)	Min. batt. %	Dup.	Cover. %	Time (s)	Min. batt. %	Dup.
EW-CVT (proposed)	**100.0**	353	**18.8**	**0.00**	**98.4**	554	19.7	1.40
Auction, no failover	**100.0**	**334**	17.4	0.90	89.2	**336**	20.0	0.75
Static partition	**100.0**	364	17.9	0.10	87.6	353	**20.3**	**0.30**

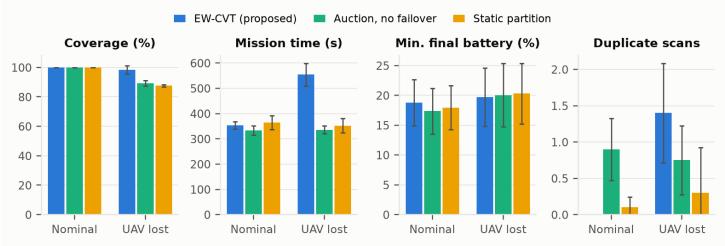


Figure 13: Allocation policies: coverage, time, battery and duplicates (bars: 95% CI).

7.6 Failure Recovery Analysis

Table 11 shows the behaviour of the swarm when UAV 2 is lost. The survivors cannot know that the UAV is gone until it is overdue at base, so failover takes minutes. It takes 231 s on average at 5 km, 143 s at 3 km and 102 s at 1.5 km, and the shorter times in smaller arenas follow from the UAVs meeting more often. The coverage stays at 98.4% or above, but the mission at 5 km grows from 353 s to 554 s. In a few seeds, sectors stay unsearched because the survivors lack the battery for a second sortie by the time they learn of the loss.

Table 11: Recovery after the loss of UAV 2. Failover: time to the first claim of its sectors.

Scenario	Cover. %	Time (s)	Failover (s)
5 km, no loss	100.0	353	n/a
5 km, loss	98.4	554	231
3 km, loss	99.0	326	143
1.5 km, loss	99.2	171	102

Lease slack

The slack trades detection speed against false alarms (Table 12, Fig. 14). For slack values of 20, 60, 120 and 240 s, the mean failover latency is 192, 231, 293 and 411 s, and the wrong suspicions fall from 11.2 of 15.6 to 0.4 of 4.7 per mission. Most suspicions concern UAVs that were fine (a landed UAV looks crashed to a peer in flight), but they cost little because released work is taken only at base.

Table 12: Effect of the lease slack after the loss of UAV 2 (5 km).

Slack (s)	Failover (s)	Mission time (s)	Coverage (%)	Duplicates	Wrong suspicions
20	192±43	518±101	98.6	1.1	11.2 of 15.6
60	231±43	554±97	98.4	1.4	8.3 of 12.6
120	293±42	587±102	98.6	0.7	3.6 of 8.0
240	411±43	698±128	98.6	0.7	0.4 of 4.7



Figure 14: Lease slack: failover latency against wrong suspicions.

Status interval

The status interval hardly affects the mission time (532 to 629 s for 2 to 60 s) until it approaches the duration of a contact. At 60 s the coverage drops to 96.4% and the duplicate scans rise from 0.6 to 2.6, while the number of control messages that are delivered falls from about 1,850 to 80.

7.7 Scaling with Swarm Size

Fig. 15 and Table 13 show the effect of the swarm size in the 5 km arena. The mission time falls from 554 s with 2 UAVs to 261 s with 12, and the energy used per UAV falls from 24.9% to 11.1%, while the recall stays at 0.980. The speed-up is sublinear because the transit from the base is a fixed cost. The small non-monotonicity between 4 and 6 UAVs is probably an artefact of how the starting territories are laid out (corners for up to 4 UAVs, a ring for more), but this has not been verified.

Table 13: Scaling with swarm size.

Swarm size	Mission time (s)	Mean energy used per UAV (%)	Recall (arena)
2	554	24.9	0.980
4	329	16.9	0.980
6	337	12.9	0.980
8	296	12.2	0.980
12	261	11.1	0.980

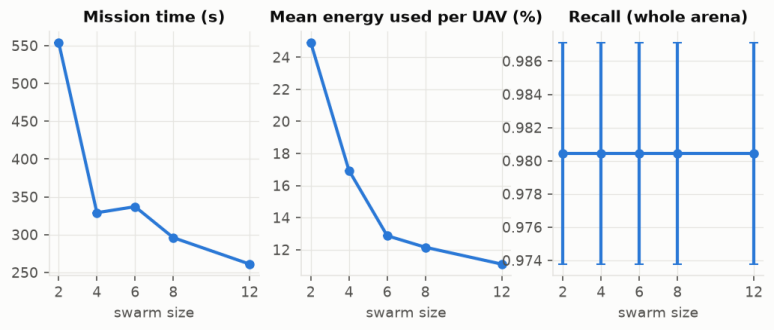


Figure 15: Mission time and energy per UAV as the swarm grows.

8 Discussion

8.1 What the Evidence Supports

Two conclusions are clearly supported. First, a sparsely connected swarm needs buffering: without the per-recipient buffers the operator knows of only 3 to 6% of the victims, and with them about 98%. Second, losing a UAV costs coverage unless the others take over its work, with a delay that is set by how long the survivors need to learn of the loss. Hysteresis matters only where partitions are recomputed from noisy readings at every tick, and energy weighting buys reserve at the price of speed.

8.2 The Price of Decentralisation

The decentralised design is robust, but it is not free. A central planner that knew at once that a UAV was lost could reassign its sectors immediately. The UAVs here cannot know, because in flight silence is normal, and so they wait for the lease of the lost UAV to expire and for the survivors to be at base. This makes the failover latency minutes and extends the mission by about 200 s at 5 km. The experiments show the shape of the trade-off: a shorter slack buys speed at the cost of wrong suspicions, which are cheap here because released work is taken only at base.

8.3 Epidemic versus Bounded Forwarding

Epidemic forwarding gives the lowest latency because every encounter spreads the alert, and it uses the most copies for the same reason. Spray-and-wait bounds the copies at $L = 3$ and pays a few seconds of latency; buffering only has a single copy and the highest latency. For a search and rescue mission, where the cost of a delayed alert is a victim who waits longer and the cost of a copy is a few bytes of storage, epidemic forwarding is the natural choice, provided that the buffer cap is large enough (Section 7).

8.4 Perception under Degraded Conditions

The fused detector keeps a high precision in every condition (0.92 to 0.99), and its weak point is the recall in thermal washout (0.39). In that condition the visible plane could help, but the network has learned to lean on thermal. This suggests training with random modality dropout, which would teach the network to use whichever plane is informative.

9 Limitations and Future Work

9.1 Limitations

1. **Simulation only.** The simulator is kinematic, with constant speeds, linear battery drain and no wind, and the network results depend on the radio model. No physical UAV has flown.
2. **Static frames.** Each sector shows one static dataset frame, so the mission-level precision and recall mostly reflect the detector on these frames and the sectors that were visited. The training and validation frames come from the same site, so the absolute numbers are probably optimistic.
3. **Perception ablation.** It masks one plane of a single fused network. Separately trained RGB-only and thermal-only detectors are needed for a proper modality comparison, and the night and washout conditions are synthetic.
4. **Simplified decentralisation.** The UAVs launch together and share the initial partition, status messages are single-hop, alerts are addressed to every UAV whether alive or not, and forwarding reads the holdings of the receiver instead of simulating the exchange on contact.

5. **Failure-detector design.** The lease detector is one of several possible designs and has not been compared with the others, and the 15% reserve and the noise levels are scenario assumptions. The comparison with the base paper is indicative only (Section 7).

9.2 Future Work

- Train separate RGB-only and thermal-only detectors (and a fused one with modality dropout) and evaluate on the full 485-frame set with AP50:95.
- Compare complete prior-work stacks across a sweep of network connectivity.
- Let status messages carry third-party state, so that failure detection does not wait for an overdue return.
- Add a camera model, wind and richer energy models, and run the control logic on the ArduPilot and ns-3 platform and finally on real hardware.

10 Conclusion

This project presented a decentralised search-and-rescue system for swarms that are mostly out of radio contact. It integrates early-fusion RGB and thermal perception, a delay-tolerant mesh with store-carry-forward relaying, and an energy-aware coordination layer without a central planner. Over 1,620 paired simulated missions, buffering was the largest effect that we measured, raising the share of victims known to the operator from 3.4% to 98% in a 5 km arena. Failover recovers 11 points of coverage after a UAV is lost, and epidemic relaying delivers alerts 7.6 s sooner than spray-and-wait at 3 km, at the price of about two more copies of each alert. The price of decentralisation is a failover latency of minutes, because no UAV can know of a loss before the missing one is overdue at base. The fused detector reaches an AP50 of 99.8 in daylight and 94.3 at night, and its main weakness is the recall under thermal washout. All results come from simulation, and the unfavourable results and the limits of the study have been reported together with the favourable ones.

References

[1] Speth, S., Gonçalves, A., Rigault, B., *et al.*: 'Deep learning with RGB and thermal images onboard a drone for monitoring operations', *J. Field Robot.*, 2022, **39**, (6), pp. 840–868 (dataset: `https://www.nii-cu-multispectral.org`).

[2] Gerkey, B.P., Matarić, M.J.: 'A formal analysis and taxonomy of task allocation in multi-robot systems', *Int. J. Robot. Res.*, 2004, **23**, (9), pp. 939–954.

[3] Grøntved, K.A.R., Lundquist, U.P.S., Christensen, A.L.: 'Decentralized multi-UAV trajectory task allocation in search and rescue applications'. Proc. IEEE Int. Conf. Advanced Robotics, Abu Dhabi, UAE, December 2023, pp. 35–41.

[4] Olfati-Saber, R., Fax, J.A., Murray, R.M.: 'Consensus and cooperation in networked multi-agent systems', *Proc. IEEE*, 2007, **95**, (1), pp. 215–233.

[5] Cortés, J., Martínez, S., Karatas, T., *et al.*: 'Coverage control for mobile sensing networks', *IEEE Trans. Robot. Autom.*, 2004, **20**, (2), pp. 243–255.

[6] Du, Q., Faber, V., Gunzburger, M.: 'Centroidal Voronoi tessellations: applications and algorithms', *SIAM Rev.*, 1999, **41**, (4), pp. 637–676.

[7] Chandra, T.D., Toueg, S.: 'Unreliable failure detectors for reliable distributed systems', *J. ACM*, 1996, **43**, (2), pp. 225–267.

[8] Hayashibara, N., Défago, X., Yared, R., *et al.*: 'The φ accrual failure detector'. Proc. IEEE Symp. Reliable Distributed Systems, Florianopolis, Brazil, October 2004, pp. 66–78.

[9] Fall, K.: 'A delay-tolerant network architecture for challenged internets'. Proc. ACM SIGCOMM, Karlsruhe, Germany, August 2003.

[10] Vahdat, A., Becker, D.: 'Epidemic routing for partially connected ad hoc networks'. Technical Report CS-2000-06, Duke University, 2000.

[11] Spyropoulos, T., Psounis, K., Raghavendra, C.S.: 'Spray and wait: an efficient routing scheme for intermittently connected mobile networks'. Proc. ACM SIGCOMM Workshop on Delay-Tolerant Networking, Philadelphia, USA, August 2005.

[12] Hwang, S., Park, J., Kim, N., *et al.*: 'Multispectral pedestrian detection: benchmark dataset and baseline'. Proc. IEEE Conf. Computer Vision and Pattern Recognition, Boston, USA, June 2015.

[13] Jia, X., Zhu, C., Li, M., *et al.*: 'LLVIP: a visible-infrared paired dataset for low-light vision'. Proc. IEEE/CVF Int. Conf. Computer Vision Workshops, Montreal, Canada, October 2021.

[14] Goodrich, M.A., Morse, B.S., Gerhardt, D., *et al.*: 'Supporting wilderness search and rescue using a camera-equipped mini UAV', *J. Field Robot.*, 2008, **25**, (1–2), pp. 89–110.

[15] Burgard, W., Moors, M., Stachniss, C., *et al.*: 'Coordinated multi-robot exploration', *IEEE Trans. Robot.*, 2005, **21**, (3), pp. 376–386.

[16] Giacomossi, L., *et al.*: 'Market-based replanning for safety-critical UAV swarms in search and rescue missions', arXiv:2606.01970, 2026.

[17] Bekmezci, İ., Sahingoz, O.K., Temel, Ş.: 'Flying ad-hoc networks (FANETs): a survey', *Ad Hoc Netw.*, 2013, **11**, (3), pp. 1254–1270.

[18] Redmon, J., Divvala, S., Girshick, R., *et al.*: 'You only look once: unified, real-time object detection'. Proc. IEEE Conf. Computer Vision and Pattern Recognition, Las Vegas, USA, June 2016.

[19] Jocher, G., Qiu, J.: 'Ultralytics YOLO11', 2024, `https://github.com/ultralytics/ultralytics`.

[20] Lloyd, S.P.: 'Least squares quantization in PCM', *IEEE Trans. Inf. Theory*, 1982, **28**, (2), pp. 129–137.

[21] Aurenhammer, F.: 'Power diagrams: properties, algorithms and applications', *SIAM J. Comput.*, 1987, **16**, (1), pp. 78–96.

[22] Kwok, A., Martínez, S.: 'Energy-balancing cooperative strategies for sensor deployment'. Proc. IEEE Conf. Decision and Control, New Orleans, USA, December 2007, pp. 6136–6141.

[23] Munir, A., Dutta, A., Parasuraman, R.: 'Energy-aware coverage planning for heterogeneous multi-robot system', arXiv:2411.02230, 2024.

[24] Gray, C.G., Cheriton, D.R.: 'Leases: an efficient fault-tolerant mechanism for distributed file cache consistency'. Proc. ACM Symp. Operating Systems Principles, Litchfield Park, USA, December 1989, pp. 202–210.

[25] Kosaka, K., Noda, Y., Yokotani, T., *et al.*: 'Implementation and evaluation of the control mechanism among distributed MQTT brokers', *IEEE Access*, 2023, **11**, doi: 10.1109/ACCESS.2023.3335273.

[26] Rappaport, T.S.: 'Wireless communications: principles and practice' (Prentice Hall, 2002, 2nd edn.).

[27] Baidya, S., Shaikh, Z., Levorato, M.: 'FlyNetSim: an open source synchronized UAV network simulator based on ns-3 and Ardupilot'. Proc. ACM Int. Conf. Modeling, Analysis and Simulation of Wireless and Mobile Systems, Montreal, Canada, October 2018, pp. 37–45.